

Document number: P0357R1
Date: 2016-09-29
Project: Programming Language C++, Library Evolution Working Group
Reply-to: Tomasz Kamiński <tomaszkam at gmail dot com>
Stephan T. Lavavej <stl at microsoft.com>
Alisdair Meredith <ameredith1 at bloomberg.net>

reference_wrapper for incomplete types

1. Introduction

This paper proposes a change in `reference_wrapper`'s specification to support incomplete types.

In addition to the necessary removal of `result_type` and related typedefs for `reference_wrapper`, the paper proposes to remove all deprecated functional components placed in the D7. Old adaptable function bindings [depr.func.adaptor.binding] section of the standard. As a consequence this paper may be viewed as a successor of [P0005R4: Adopt not_fn from Library Fundamentals 2 for C++17](#).

2. History

2.1. Revision 1

- Minor corrections and clarifications to paper text. Clarified relation to [P0091: Template argument deduction for class templates](#)
- Updated wording to refer to [N4606](#) (C++ Working Draft, 2016-07-12) and included content of removed subclause.
- Included proposed feature testing recommendation macro.
- Corrected included implementation for the `operator()` of `reference_wrapper`, so it is now `const` qualified.
- Extended implementability section to discuss details of instantiation of `reference_wrapper` and `operator()` member.

3. Motivation and Scope

`std::reference_wrapper` is a utility class created to allow references to be used in interfaces that were designed to pass objects by value. However the design of the standard component has a major drawback, when compared to the alternative solutions based on the use of the raw pointers of boost version of this component: the referenced type is required to be complete. As a consequence, depending on the context use of `reference_wrapper` may increase compilation time by adding a new definition, or even be impossible in the case when the definition of the class is not available to the programmer.

Moreover `std::reference_wrapper` specializations are recognized by standard factory functions, like: `std::make_pair`, `std::make_tuple` or `std::bind` which allow the programmer to create pairs of references by use of:

```
auto p = std::make_pair(std::ref(t), std::ref(u));
```

Use of this feature, not only avoids cumbersome specification of the type, but also eliminates the possibility of encountering dangling reference problems that it may introduce. For example in the case of the following definition:

```
std::pair<std::string const&, int> p("test", 10);
```

every use of `p.first` leads to undefined behaviour caused by reading a dangling reference. Despite all of the above advantages, programmers are still forced to use `pair<T&, U&>`, when at least one of types the `T` or `U` is incomplete.

Furthermore this problem is not addressed by inclusion of the [P0091: Template argument deduction for class templates](#), as implicit deduction guides synthesized from `pair` and `tuple` constructors will not deduce reference types. And if special treatment of `std::reference_wrapper<T>` will be reintroduced, the same limitations will apply (`T` needs to be a complete type).

4. Design Decisions

Supporting incomplete types in `reference_wrapper` is currently impossible because the implementation is required to check the template parameter for presence of `result_type` and related nested types. As a consequence support for these (now deprecated) features needs to be removed from the standard.

4.1. Removal of deprecated functional components

Despite the fact that support for incomplete types requires only removal of support for *weak result type* and argument typedefs for `reference_wrapper`, this paper follows the original direction of [P005R4: Adopt not_fn from Library Fundamentals 2 for C++17](#) and proposes removal of all deprecated function bindings. As this paper is targeted for the next standard after C++17, it also follows the committee guideline to introduce a period of deprecation before removal.

In addition, the feature proposed with the paper conflicts with support for the old function binding protocol and vendors will no longer be allowed to provide required typedefs in their `std::reference_wrapper` implementations.

4.2. Support for `is_transparent`

In C++14, another protocol based on the presence of the `is_transparent` nested type was introduced, to indicate that a given functor enables heterogeneous lookup for associative container. As in the case of `result_type` implementation of this protocol in exact form for `reference_wrapper<T>` would reintroduce requirement of completeness of τ template parameter.

Despite the fact that support for incomplete types and heterogeneous container lookup in `reference_wrapper` may look incompatible, there is the possibility to provide both of them, via an alternative design that relies on a metafunction instead of a nested type, as proposed in [P0046R1: Change is_transparent to metafunction \(Revision 1\)](#).

5. Impact On The Standard

This proposal depends on the deprecation of the `result_type` and related typedefs in C++17, so they can be removed in an upcoming standard after a period of deprecation.

Nothing depends on this proposal.

6. Proposed Wording

The proposed wording changes refer to [N4606](#) (C++ Working Draft, 2016-07-12).

Apply following changes to paragraph 17.6.4.3.1 Zombie names [zombie.names]:

In namespace `std`, the following names are reserved for previous standardization: `auto_ptr`, `binary_function`, `binary_negate`, `bind1st`, `bind2nd`, `binder1st`, `binder2nd`, `const_mem_fun1_ref_t`, `const_mem_fun1_t`, `const_mem_fun_ref_t`, `const_mem_fun_t`, `mem_fun1_ref_t`, `mem_fun1_t`, ~~`mem_fun_ref_t`~~, ~~`mem_fun`~~, `mem_fun_ref`, ~~`mem_fun_ref_t`~~, `mem_fun_t`, ~~`mem_fun`~~, `not1`, `not2`, `pointer_to_binary_function`, `pointer_to_unary_function`, `ptr_fun`, `random_shuffle`, ~~`and_unary_function`~~, `and_unary_negate`.

It is unspecified whether function objects in the C++ standard library additionally provide the following typedefs: `result_type`, `argument_type`, `first_argument_type`, and `second_argument_type`.

At the end of the section 20.14.4 Class template `reference_wrapper` [refwrap]:

The template parameter τ of `reference_wrapper` may be an incomplete type.

Apply following changes to paragraph 20.14.4.4 `reference_wrapper` invocation [refwrap.invoke]:

```
template <class... ArgTypes>
    result_of_t<T&(ArgTypes&&... )>
    operator()(ArgTypes&&... args) const;
```

Returns: `INVOKE(get(), std::forward<ArgTypes>(args)...) .` ([func.require] 20.12.2).

Remarks: If τ is an incomplete type, the program is ill-formed.

At the beginning of the paragraph 20.14.4.5 `reference_wrapper` helper functions [refwrap.helpers]:

The template parameter τ of the following `ref` and `cref` function templates may be an incomplete type.

Under Annex C Compatibility [diff] add new subclause after C.4 C++ and ISO C++ 2014 [diff.cpp14]:

This subclause lists the differences between C++ and ISO C++ 2017 (TBD), by the chapters of this document.

C.x.1 Clause 20: general utilities library**[diff.cpp17.utilities]**

Change: The typedefs `result_type`, `argument_type`, `first_argument_type`, and `second_argument_type` might not be defined by some function objects.

Rationale: Superseded by new features, including `decltype` and forwarding references.

Effect on original feature: Valid C++ 2017 code that uses these typedefs may fail to compile in this International Standard.

Change: The class templates `unary_negate` and `binary_negate` and the function templates `not1` and `not2` might not be defined.

Rationale: Superseded by new features, including generic lambdas and the function template `not_fn`.

Effect on original feature: Valid C++ 2017 code that uses these class templates and function templates may fail to compile in this International Standard.

Remove subclause D8. Old adaptable function bindings [depr.func.adaptor.binding] entirely:

~~D.8 Old Adaptable Function Bindings~~**~~[depr.func.adaptor.binding]~~****~~D.8.1 Weak Result Types~~****~~[depr.weak.result_type]~~**

1. If a call wrapper (20.9.1) has a *weak result type*, the type of its member type `result_type` is based on the type `T` of the wrapper's target object (20.9.1):
 1. if `T` is a pointer to function type, `result_type` shall be a synonym for the return type of `T`;
 2. if `T` is a pointer to member function, `result_type` shall be a synonym for the return type of `T`;
 3. if `T` is a class type and the qualified-id `T::result_type` is valid and denotes a type (14.8.2), then `result_type` shall be a synonym for `T::result_type`;
 4. otherwise `result_type` shall not be defined.

~~D.8.2 Typedefs to Support Function Binders~~**~~[depr.func.adaptor.typedefs]~~**

1. To enable old function adaptors to manipulate function objects that take one or two arguments, many of the function objects in this standard correspondingly provide typedefs `argument_type` and `result_type` for function objects that take one argument and `first_argument_type`, `second_argument_type`, and `result_type` for function objects that take two arguments.
2. The following member names are defined in addition to names specified in Clause 20:

```
namespace std {
  template<class T> struct owner_less<shared_ptr<T>> {
    typedef bool result_type;
    typedef shared_ptr<T> first_argument_type;
    typedef shared_ptr<T> second_argument_type;
  };

  template<class T> struct owner_less<weak_ptr<T>> {
    typedef bool result_type;
    typedef weak_ptr<T> first_argument_type;
    typedef weak_ptr<T> second_argument_type;
  };

  template<class T> class reference_wrapper {
  public:
    typedef see below result_type; // not always defined
    typedef see below argument_type; // not always defined
    typedef see below first_argument_type; // not always defined
    typedef see below second_argument_type; // not always defined
```

```

—};

—template <class T = void> struct plus {
—    typedef T first_argument_type;
—    typedef T second_argument_type;
—    typedef T result_type;
—};

—template <class T = void> struct minus {
—    typedef T first_argument_type;
—    typedef T second_argument_type;
—    typedef T result_type;
—};

—template <class T = void> struct multiplies {
—    typedef T first_argument_type;
—    typedef T second_argument_type;
—    typedef T result_type;
—};

—template <class T = void> struct divides {
—    typedef T first_argument_type;
—    typedef T second_argument_type;
—    typedef T result_type;
—};

—template <class T = void> struct modulus {
—    typedef T first_argument_type;
—    typedef T second_argument_type;
—    typedef T result_type;
—};

—template <class T = void> struct negate {
—    typedef T argument_type;
—    typedef T result_type;
—};

—template <class T = void> struct equal_to {
—    typedef T first_argument_type;
—    typedef T second_argument_type;
—    typedef bool result_type;
—};

—template <class T = void> struct not_equal_to {
—    typedef T first_argument_type;
—    typedef T second_argument_type;
—    typedef bool result_type;
—};

—template <class T = void> struct greater {
—    typedef T first_argument_type;
—    typedef T second_argument_type;
—    typedef bool result_type;
—};

—template <class T = void> struct less {
—    typedef T first_argument_type;
—    typedef T second_argument_type;
—    typedef bool result_type;
—};

—template <class T = void> struct greater_equal {
—    typedef T first_argument_type;
—    typedef T second_argument_type;
—    typedef bool result_type;
—};

—template <class T = void> struct less_equal {
—    typedef T first_argument_type;
—    typedef T second_argument_type;
—    typedef bool result_type;
—};

—template <class T = void> struct logical_and {
—    typedef T first_argument_type;
—    typedef T second_argument_type;
—    typedef bool result_type;
—};

```

```

—template <class T = void> struct logical_or {
—    typedef T first_argument_type;
—    typedef T second_argument_type;
—    typedef bool result_type;
—};

—template <class T = void> struct logical_not {
—    typedef T argument_type;
—    typedef bool result_type;
—};

—template <class T = void> struct bit_and {
—    typedef T first_argument_type;
—    typedef T second_argument_type;
—    typedef T result_type;
—};

—template <class T = void> struct bit_or {
—    typedef T first_argument_type;
—    typedef T second_argument_type;
—    typedef T result_type;
—};

—template <class T = void> struct bit_xor {
—    typedef T first_argument_type;
—    typedef T second_argument_type;
—    typedef T result_type;
—};

—template <class T = void> struct bit_not {
—    typedef T argument_type;
—    typedef T result_type;
—};

—template<class R, class... ArgTypes>
—class function<R(ArgTypes...)> {
—public:
—    typedef T1 argument_type; // only if sizeof...(ArgTypes) == 1 and
—                                // the type in ArgTypes is T1
—    typedef T1 first_argument_type; // only if sizeof...(ArgTypes) == 2 and
—                                    // ArgTypes contains T1 and T2
—    typedef T2 second_argument_type; // only if sizeof...(ArgTypes) == 2 and
—                                    // ArgTypes contains T1 and T2
—};
}

```

3. `reference_wrapper<T>` has a weak result type (D.x.1). If `T` is a function type, `result_type` shall be a synonym for the return type of `T`.
4. The template specialization `reference_wrapper<T>` shall define a nested type named `argument_type` as a synonym for `T1` only if the type `T` is any of the following:
 1. —a function type or a pointer to function type taking one argument of type `T1`
 2. —a pointer to member function `R T0::f cv` (where `cv` represents the member function's cv-qualifiers); the type `T1` is `cv T0*`
 3. —a class type where the qualified-id `T::argument_type` is valid and denotes a type (14.8.2); the type `T1` is `T::argument_type`.
5. The template instantiation `reference_wrapper<T>` shall define two nested types named `first_argument_type` and `second_argument_type` as synonyms for `T1` and `T2`, respectively, only if the type `T` is any of the following:
 1. —a function type or a pointer to function type taking two arguments of types `T1` and `T2`
 2. —a pointer to member function `R T0::f(T2) cv` (where `cv` represents the member function's cv-qualifiers); the type `T1` is `cv T0*`
 3. —a class type where the qualified-ids `T::first_argument_type` and `T::second_argument_type` are both valid and both denote types (14.8.2); the type `T1` is `T::first_argument_type` and the type `T2` is `T::second_argument_type`.
6. For all object types `Key` for which there exists a specialization `hash<Key>`, and for all enumeration types (7.2) `Key`, the instantiation `hash<Key>` shall provide two nested types, `result_type` and `argument_type`, which shall be synonyms for `size_t` and `Key`, respectively.
7. The forwarding call wrapper `g` returned by a call to `bind(f, bound_args...)` (20.9.10.3) shall have a weak result type (D.x.1).
8. The forwarding call wrapper `g` returned by a call to `bind<R>(f, bound_args...)` (20.9.10.3) shall have a nested type `result_type` defined as a synonym for `R`.
9. The simple call wrapper returned from a call to `mem_fn(pm)` shall have a nested type `result_type` that is a synonym

for the return type of `pm` when `pm` is a pointer to member function:

10. The simple call wrapper returned from a call to `mem_fn(pm)` shall define two nested types named `argument_type` and `result_type` as synonyms for `cv T*` and `Ret`, respectively, when `pm` is a pointer to member function with `cv`-qualifier `cv` and taking no arguments, where `Ret` is `pm`'s return type.
11. The simple call wrapper returned from a call to `mem_fn(pm)` shall define three nested types named `first_argument_type`, `second_argument_type`, and `result_type` as synonyms for `cv T*`, `T1`, and `Ret`, respectively, when `pm` is a pointer to member function with `cv`-qualifier `cv` and taking one argument of type `T1`, where `Ret` is `pm`'s return type.
12. The following member names are defined in addition to names specified in Clause 23:

```
namespace std {
template <class Key, class T, class Compare = less<Key>,
          class Allocator = allocator<pair<const Key, T>>>
class map {
public:
    class value_compare {
    public:
        typedef bool result_type;
        typedef value_type first_argument_type;
        typedef value_type second_argument_type;
    };
};

template <class Key, class T, class Compare = less<Key>,
          class Allocator = allocator<pair<const Key, T>>>
class multimap {
public:
    class value_compare {
    public:
        typedef bool result_type;
        typedef value_type first_argument_type;
        typedef value_type second_argument_type;
    };
};
}
```

D.8.3 Negators

[depr.negators]

1. The header `<functional>` has the following additional declarations:

```
namespace std {
template <class Predicate> class unary_negate;
template <class Predicate>
constexpr unary_negate<Predicate> not1(const Predicate&);
template <class Predicate> class binary_negate;
template <class Predicate>
constexpr binary_negate<Predicate> not2(const Predicate&);
}
```

2. Negators `not1` and `not2` take a unary and a binary predicate, respectively, and return their complements (5.3.1):

```
template <class Predicate>
class unary_negate {
public:
    constexpr explicit unary_negate(const Predicate& pred);
    constexpr bool operator()(const typename Predicate::argument_type& x) const;
    typedef typename Predicate::argument_type argument_type;
    typedef bool result_type;
};
```

3. `operator()` returns `!pred(x)`:

```
template <class Predicate>
constexpr unary_negate<Predicate> not1(const Predicate& pred);
```

4. *Returns:* `unary_negate<Predicate>(pred)`:

```
template <class Predicate>
class binary_negate {
public:
    constexpr explicit binary_negate(const Predicate& pred);
    constexpr bool operator()(const typename Predicate::first_argument_type& x,
                             const typename Predicate::second_argument_type& y) const;
    typedef typename Predicate::first_argument_type first_argument_type;
    typedef typename Predicate::second_argument_type second_argument_type;
```

```
—typedef bool result_type;  
—};
```

5. ~~operator()~~ ~~returns~~ ~~!pred(x,y);~~

```
template<class Predicate>  
—constexpr binary_negate<Predicate> not2(const Predicate& pred);
```

6. *Returns:* ~~binary_negate<Predicate>(pred);~~

7. Feature-testing recommendation

For the purposes of SG10, we recommend the macro name `__cpp_lib_reference_wrapper` with value `20YMM` representing publication date, to be defined in the `<functional>` header. The intent is to allow reuse of the same macro to indicate presence of the original feature from C++11 standard.

Usage example:

```
template<typename T, typename U>  
auto my_tie(T& t, U & u)  
{  
#if __cpp_lib_reference_wrapper >= 20YMM  
    return std::make_pair(std::ref(t), std::ref(u));  
#else  
    return std::pair<T&, U&>(t, u);  
#endif  
}
```

8. Implementability

Without requirement to conditionally support `result_type` and related typedefs, straightforward implementation provides support for incomplete types.

```
template<typename T>  
class reference_wrapper  
{  
    T* ptr;  
  
public:  
    using type = T;  
  
    reference_wrapper(T& val) noexcept  
        : ptr(std::addressof(val))  
    {}  
  
    reference_wrapper(T&&) = delete;  
  
    T& get() const noexcept { return *ptr; }  
    operator T&() const noexcept { return *ptr; }  
  
    template<typename... Args>  
    auto operator()(Args&&... args) const  
        -> std::result_of_t<T&(Args...)>  
    { return std::invoke(*ptr, std::forward<Args>(args)...); }  
};
```

Careful reader may notice, that the `operator()` requires template parameter `T` to be a complete type, and this requirement is not only limited to definition of the function, but also its declaration, that uses `std::result_of_t<T&(Args...)>` to specify return type. However, call `operator` is an template function member of the class and its declaration will not be instantiated during the instantiation of enclosing `reference_wrapper` specialization, as the `Args` template parameter pack are not know at this point. As a consequence the user is allowed to use an object of `reference_wrapper<T>` with `T` being an incomplete type, unless it is actually called.

9. Acknowledgements

Special thanks and recognition goes to Sabre (<http://www.sabre.com>) for supporting the production of this proposal, and for sponsoring Tomasz Kamiński's trip to the Oulu for WG21 meeting.

10. References

1. Alisdair Meredith, Stephan T. Lavavej, Tomasz Kamiński, "Adopt `not_fn` from Library Fundamentals 2 for C++17", (P0005R4, <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0005r4.html>)
2. Tomasz Kamiński, "Change `is_transparent` to metafunction (Revision 1)", (P0046R1, <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0046r1.html>)
3. Mike Spertus, Faisal Vali, Richard Smith, "Template argument deduction for class templates (Rev. 6)", (P0091R3, <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0091r3.html>)
4. Richard Smith, "Working Draft, Standard for Programming Language C++" (N4606, <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/n4606.pdf>)